

CampusEats Orders REST Service

Assignment 5 · HTTP Methods, Headers, Caching & Safe Retries

GROUP ID	GROUP LEADER
12	Subham Kumar Mohanty — 20252651057
TEAM MEMBERS	ROLL / REGISTRATION NUMBERS
Harsh N Shende	20252651023
Abhishek Jadhao	20252651003
Mahi Verma	20252651032
Rishabh Mishra	20252651041

Submission overview

This report documents the Group 12 implementation of Assignment 5 for the CampusEats Orders API. The work applies correct HTTP method semantics, query parameters for collection reads, sub-resources for non-CRUD actions, request and response header rules, authentication, CORS, caching, rate-limit signalling, ETags, conditional requests, and retry protection.

The service is implemented in Python with a WSGI-based server and is designed to demonstrate the required HTTP behaviour without relying on external runtime packages.

Part A · HTTP methods and message design

The endpoint design below distinguishes read-only operations from state-changing operations and identifies where idempotency or conditional requests are needed.

Endpoint map

Action	Request	Safe	Idempotent	Implementation note
List / filter / sort / paginate	GET /orders?status=&sort=&page=&pageSize=	Yes	Yes	Query parameters keep this a pure read.
Read one order	GET /orders/{orderId}	Yes	Yes	Returns an ETag and cache policy; supports conditional GET.
Create order	POST /orders	No	No*	201 Created + Location. An Idempotency-Key makes a retry return the original result.
Change delivery address	PATCH /orders/{orderId}	No	Yes	Uses If-Match to avoid lost updates.
Delete order	DELETE /orders/{orderId}	No	Yes	204 No Content; protected by If-Match.
Request cancellation	POST /orders/{orderId}/cancellation	No	No	Action is modelled as a POST sub-resource and returns 202.
Discover methods	OPTIONS /orders/{orderId}	Yes	Yes	Returns Allow and participates in CORS preflight.

Client fallback: A constrained client may send POST with X-HTTP-Method-Override: PATCH or DELETE when it cannot transmit those verbs directly.

Part B · Headers on every message

The service uses headers to communicate media type, authentication, caching, concurrency, rate limits, CORS permissions, and security policy.

Request / response header matrix

Endpoint	Request headers accepted / required	Key response headers and status
GET /orders	Authorization; optional Accept and Accept-Encoding	200; JSON Content-Type, private cache policy, optional gzip, rate-limit, CORS and security headers.
POST /orders	Authorization, JSON Content-Type, Idempotency-Key, Accept	201 Created; Location, ETag, no-store, rate-limit, CORS and security headers.
GET /orders/{id}	Authorization; optional If-None-Match and Accept	200 or 304; ETag, private max-age=60, CORS and security headers.
PATCH /orders/{id}	Authorization, Content-Type, required If-Match	200 or 412; new ETag and no-store.
DELETE /orders/{id}	Authorization, required If-Match	204 or 412; no-store.
POST /orders/{id}/cancellation	Authorization, JSON Content-Type	202, 409 or 422; no-store.
OPTIONS	Origin and Access-Control-Request-Method for browser preflight	204; Allow and Access-Control-* headers.

Security and protocol details

Protected routes require `Authorization: Bearer <token>`; an absent or empty credential is treated as 401 Unauthorized. JSON bodies use `Content-Type: application/json`, and unsupported requested representations are rejected with 406 Not Acceptable.

The service exposes rate-limit information through `X-RateLimit-Limit` and `X-RateLimit-Remaining`. When a client's budget is exhausted, the API uses 429 Too Many Requests together with `Retry-After`.

CORS responses include `Access-Control-Allow-Origin` and the `OPTIONS` preflight path. Security responses also include `X-Content-Type-Options: nosniff` and `Strict-Transport-Security`.

Caching rules

Single-order GET responses carry an ETag that changes whenever the representation changes. Read responses may be privately cached, while mutation responses use `Cache-Control: no-store`.

Large JSON responses can use gzip when the client advertises support through `Accept-Encoding`.

Part C · Caching and safe retries

The retry plan separates three different concerns: repeating a creation request safely, avoiding unnecessary transfer of unchanged data, and preventing stale updates.

Retry / concurrency strategy

Endpoint	Risk	Mechanism	How it helps
POST /orders	A lost response can make the client submit the order again, potentially creating duplicate work or charges.	Idempotency-Key	The first result is associated with the key. A repeat returns that result rather than creating another order.
GET /orders/{id}	An unchanged representation may be downloaded repeatedly.	If-None-Match	A matching ETag produces 304 Not Modified without another response body.
PATCH /orders/{id}	Two clients may update from different versions of the same order.	If-Match	A stale ETag is rejected with 412, so a newer change is not silently overwritten.
POST /orders/{id}/cancellation	A repeated action can target an order already in a terminal state.	409 / 422 state checks	Conflict and invalid lifecycle transitions are explicitly rejected.

Why these mechanisms are different

An Idempotency-Key is tied to the business operation and is therefore appropriate for order creation. If-None-Match is about whether a cached representation is still current. If-Match is a concurrency check: the write proceeds only when the client is operating on the expected representation.

Conditional GET example

```
GET /orders/ord-f8a326caa1a1 HTTP/1.1
Authorization: Bearer demo-token
If-None-Match: "0aa5da47aa253af6"
Accept: application/json
```

```
HTTP/1.0 304 Not Modified
ETag: "0aa5da47aa253af6"
```

Conditional write example

```
PATCH /orders/ord-f8a326caa1a1 HTTP/1.1
Authorization: Bearer demo-token
If-Match: "stale-etag-99999"
Content-Type: application/json
```

```
HTTP/1.0 412 Precondition Failed
```

The first exchange avoids retransmitting an unchanged representation. The second prevents the classic lost-update situation.

Idempotent cancellation behaviour

Cancellation is represented as an action sub-resource rather than a verb embedded in a URL. A repeated request is checked against the order's lifecycle state; a conflict is reported as 409, while a structurally valid but semantically disallowed state transition is represented as 422.

Part D · Complete HTTP exchange

The following example shows a complete order-creation message, including the request line, host, negotiation, authentication, idempotency token, response status, and important response headers.

Request · HTTP/1.1

```
POST /orders HTTP/1.1
Host: 127.0.0.1:8001
User-Agent: curl/8.5.0
Accept: application/json
Content-Type: application/json
Authorization: Bearer demo-token
Idempotency-Key: a5-demo-key-001
Content-Length: 119
```

```
{"customerId":"stu-102","items":[{"menuItemId":"menu-42","quantity":2}],"deliveryAddress":"Hostel A, Room 210"}
```

Response · development WSGI server

```
HTTP/1.0 201 Created
Date: Tue, 22 Sep 2026 03:06:53 GMT
Server: WSGIServer/0.2 CPython/3.12.3
Content-Type: application/json
Access-Control-Allow-Origin: *
Access-Control-Allow-Methods: GET, POST, PATCH, ...
Access-Control-Allow-Headers: Authorization, ...
Access-Control-Expose-Headers: Location, ETag, ...
Vary: Accept, Accept-Encoding, Origin
X-Content-Type-Options: nosniff
Strict-Transport-Security: max-age=31536000
X-RateLimit-Limit: 50
X-RateLimit-Remaining: 49
Location: /orders/ord-f8a326caa1a1
Cache-Control: no-store
ETag: "0aa5da47aa253af6"
Content-Length: 209
```

```
{"id":"ord-f8a326caa1a1","customerId":"stu-102","items":[{"menuItemId":"menu-42","quantity":2}],"deliveryAddress":"Hostel A, Room 210"}
```

Reading the exchange

201 Created confirms that a new order resource was created. **Location** identifies that resource. The **ETag** fingerprints its representation, while **Cache-Control: no-store** keeps the mutation out of caches.

Access-Control-* headers make the response usable by browser clients subject to the configured CORS policy. The two rate-limit headers communicate the current request budget.

Protocol note: the client request is written as HTTP/1.1, while the standard Python development WSGI server shown here responds using HTTP/1.0 close-on-finish semantics. A production deployment behind Nginx/TLS can provide HTTP/1.1 or HTTP/2.

Representative failure statuses

Status	Example condition	Meaning
400 Bad Request	Malformed JSON or missing required request fields	The request does not satisfy the expected request structure.
401 Unauthorized	Missing or empty Authorization token	Authentication information was not supplied correctly.
404 Not Found	Requested order ID does not exist	The target resource cannot be found.
409 Conflict	Invalid conflict with current order state	The requested action conflicts with the resource's current state.
412 Precondition Failed	If-Match does not match current ETag	The client attempted a conditional write from a stale version.
422 Unprocessable Content	Valid request but invalid lifecycle transition	The structure is valid, but domain rules reject the operation.

Assignment questions · 1 to 4

1. Three endpoints: method, success status and key response header

POST /orders returns **201 Created**; Location is important because the generated order URI is not known to the client beforehand. **GET /orders/{orderId}** returns **200 OK**; ETag identifies the current representation and enables conditional reads and writes. **DELETE /orders/{orderId}** returns **204 No Content**; Cache-Control: no-store prevents an intermediary from retaining a stale deleted representation.

2. Safe versus idempotent operations

The safe endpoints are GET /orders, GET /orders/{id}, and OPTIONS. The idempotent operations include GET, OPTIONS, PATCH, and DELETE. POST /orders is neither safe nor naturally idempotent, so the service requires an Idempotency-Key for retry protection. Cancellation remains a state-changing POST action and is controlled through lifecycle checks.

3. ETag, 304 and 412

Example ETag: "0aa5da47aa253af6". A GET carrying If-None-Match: "0aa5da47aa253af6" can return **304 Not Modified**, avoiding another representation transfer. A PATCH using an outdated If-Match value such as "stale-etag-99999" returns **412 Precondition Failed**, stopping a stale editor from overwriting newer data.

4. 400 versus 422

400 Bad Request: POST /orders with {"customerId": "stu-102"} when required fields such as items and deliveryAddress are missing. **422 Unprocessable Content**: POST /orders/{id}/cancellation when the order is already cancelled or delivered. The distinction is structural validity versus business-state validity: 400 concerns the request shape, while 422 describes a well-formed request that the domain rules cannot accept.

Practical interpretation

Together, these cases show why HTTP status codes and headers carry more than transport information. Location tells the client where a newly created resource lives; ETag-based conditions protect representation consistency; and 400/422 separate malformed input from a valid request that is not allowed by the current application state.

The method design also keeps non-CRUD actions such as cancellation out of the URL verb itself and instead expresses the action through POST on a sub-resource.

Assignment questions · 5 to 8

5. Cross-origin request blocked in the browser

The browser enforces the Same-Origin Policy. The server may still log a successful 200 response, but browser JavaScript cannot read the response unless the server grants cross-origin access. The relevant response header is `Access-Control-Allow-Origin`, accompanied by the appropriate `OPTIONS` preflight response when required.

6. Cacheable response versus no-store response

A single-order GET may use `Cache-Control: private, max-age=60` because a student's browser can reuse a recent private representation and use the ETag to check freshness. Order creation and updates use `Cache-Control: no-store` because these are mutations and should not be replayed from an HTTP cache.

7. When POST can replace GET for a search

POST can be considered when the search criteria are too large or complex to fit comfortably in a URL, such as a large collection of identifiers or a deeply nested filter structure. It may also keep sensitive criteria out of URL history and ordinary URL logging. The trade-off is that POST loses several natural GET advantages, including straightforward URL bookmarking, link sharing, browser navigation semantics, and normal HTTP cacheability.

8. Meaning of Location on 201 and 3xx

For **201 Created**, the Location header identifies the newly created resource. For a **3xx redirect**, Location identifies the URI to which the client should redirect or continue the request.

Verification evidence

```
$ python3 run_tests.py
Running 15 test cases for CampusEats Orders API (Assignment 5)...
[PASS] test_cancellation_sub_resource
[PASS] test_conditional_delete_and_412
[PASS] test_conditional_get_returns_304
[PASS] test_conditional_patch_update_and_412
[PASS] test_content_negotiation_unsupported_accept_returns_406
[PASS] test_create_returns_201_location_etag_and_security_headers
[PASS] test_gzip_compression_on_large_payload
[PASS] test_idempotent_repeat_returns_original_order
[PASS] test_invalid_query_params_return_400
[PASS] test_malformed_body_uses_problem_shape
[PASS] test_method_override_patch_and_delete
[PASS] test_missing_token_returns_401_and_unknown_returns_404
[PASS] test_options_discovery_and_cors
[PASS] test_query_params_filtering_sorting_and_pagination
[PASS] test_rate_limiting_signal
=====
Results: 15 passed, 0 failed in 0.009s
=====

$ python3 validate_openapi.py openapi.yaml
OpenAPI document parsed successfully.
Structural checks completed: 0 errors.
```

Submission note

This Group 12 report is a separately formatted and reworded version of the Assignment 5 material. It retains the complete six-page scope: method mapping, header policy, safe-retry plan, full HTTP exchange, all eight assignment answers, and verification evidence.

Group 12 · Subham Kumar Mohanty · Harsh N Shende · Abhishek Jadhao · Mahi Verma · Rishabh Mishra